Hochschule Konstanz
Fakultät Informatik

Software Architecture

10 - Reactive Streams

Prof. Dr. Marko Boger

Evaluation of Members

So far we have seen the following member declarations

- var
  - to be avoided, mutable data
- val
  - immutable data
- def
  - functions

Var, val and def are actually very similar, they each can call a function to deliver a result.

The difference is really only in the point in time when the function is called or evaluated. Vals are evaluated at the creation of an object instance, where as var and def are evaluated on every call.

Lazy Evaluation

There is a fourth form of declaration of a member

- lazy val

Lazy evaluation is not executed at creation time, but when the value is first requested. This might be a lot later.

This can be used to move expensive execution time to a later point in time.

```
val x = someExpensiveComputation // executed at creation of object
lazy val y = someExpensiveComputation // executed when called first time
```

But it has further important implications ...

Infinite Sequences

Using lazy evaluation we can define infinite structures, for example by an infinite recursion, but since only requested elements are actually calculated, it behaves finite.

Streams in the standard Scala collections are lazily evaluated Lists.

Here is an example:

```
scala> Stream.from(1) take 10 toList
res23: List[Int] = List(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

scala> Stream.from(1,2) take 10 toList
res25: List[Int] = List(1, 3, 5, 7, 9, 11, 13, 15, 17, 19)
```

Infinite Sequences

Here is another example, the Fibonacci number:

```
def fib1: Stream[Int] = {
 def loop(a: Int, b: Int): Stream[Int] = a  #:: loop(b, a + b)
 loop(0, 1)
}

fib1 take 20 toList
res0: List[Int] = List(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181)
```

The Stream API

Streams are a collection, just like list, and a monad. So it has map, filter, head, tail, cons as methods. Short-form of cons is #::

```
def fib2:Stream[Int] ={
 def loop(a: Int, b:Int): Stream[Int] = Stream.cons(a,loop(b,a+b))
 loop(0,1)
}
fib2 take 20 toList

def fib3(a: Int = 0, b: Int = 1): Stream[Int] = a #:: fib3(b, a+b)
fib3() take 20 toList

def fib4(a: Int , b: Int): Stream[Int] = a #:: fib4(b, a+b)
fib4(0,1) take 20 toList

val fib5: Stream[Int] = 0 #:: fib5.scanLeft(1)(_ + _)
fib5 take 20 toList

val fib6:Stream[Int] = 0 #:: 1 #:: (fib6 zip fib6.tail).map{ t => t._1 + t._2}
fib6 take 20 toList
```

Another Example: Prime Numbers as Stream

Here is an algorithm for the Sieve of Eratosthemes:

```
scala> def primeStream(s: Stream[Int]): Stream[Int] = s.head #:: primeStream(s.tail filter { _ %
s.head != 0 })
primeStream: (s: Stream[Int])Stream[Int]

scala> val primes = primeStream(Stream.from(2))
primes: Stream[Int] = Stream(2, ?)

scala> primes take 10 foreach println
2
3
5
7
11
13
17
19
23
29
```

Reactive Streams

Reactive Streams extend on the idea of Streams by adding the concept of **back pressure**.

Reactive Streams is a specification, developed by a consortium, including Lightbend, Netflix, Twitter, Oracle and RedHat. Work started in 2013, spec was released 2015.

The Specification includes a TCK (Technology Compatibility Kit), a test suite.

Akka Streams is an implementation of Reactive Streams and was first to pass the TCK. There are now several implementations and compatible technologies.

The Reactive Streams Spec

The Specification defines 4 concepts

1. Publisher
2. Subscriber
3. Subscription
4. Processor

and rules for them

https://github.com/reactive-streams/reactive-streams-jvm/blob/v1.0.3/README.md#specification

Problem: Overflowing Buffer

A problem in asynchronous messaging is the possibility of overflowing buffers.

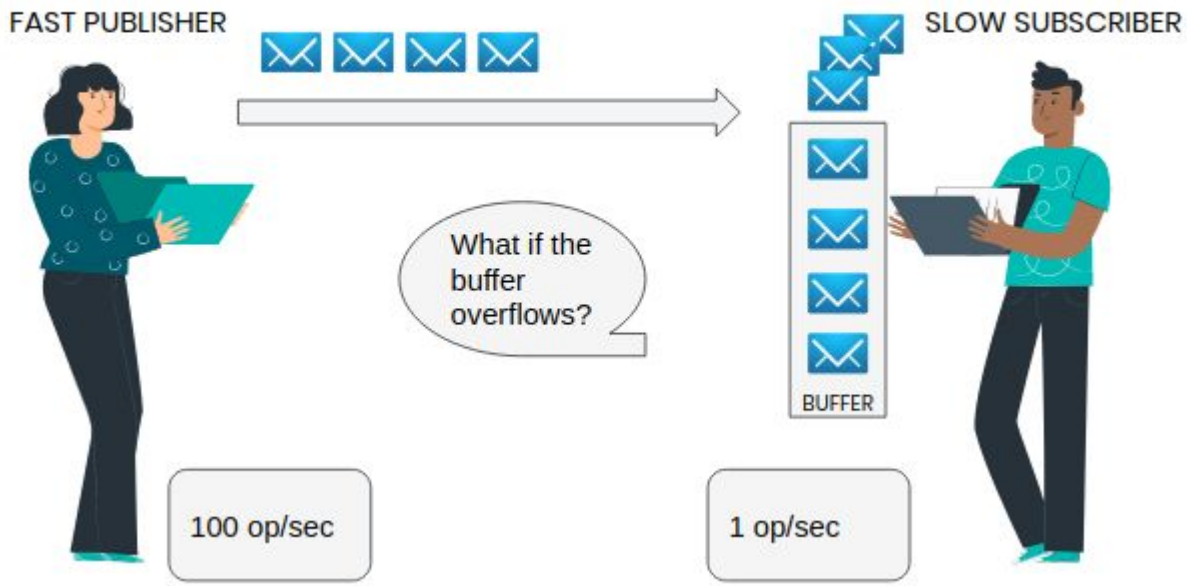
For example, an Actor has a message inbox. If messages come in faster than the actor can process, then eventually it overflows.

The actor could drop the overflow, increase buffer, request resends, or halt all together.

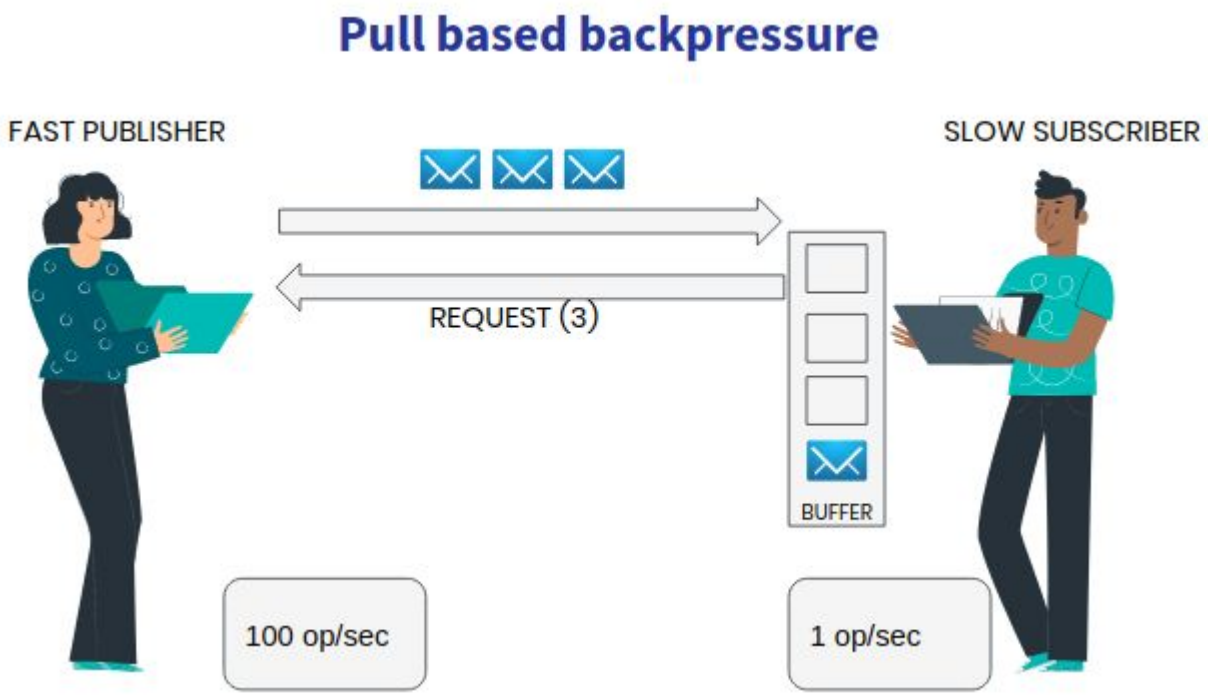
Or it can inform the sender to slow down. That is the idea of back pressure.

Publisher and Subscriber

Potential Overflow



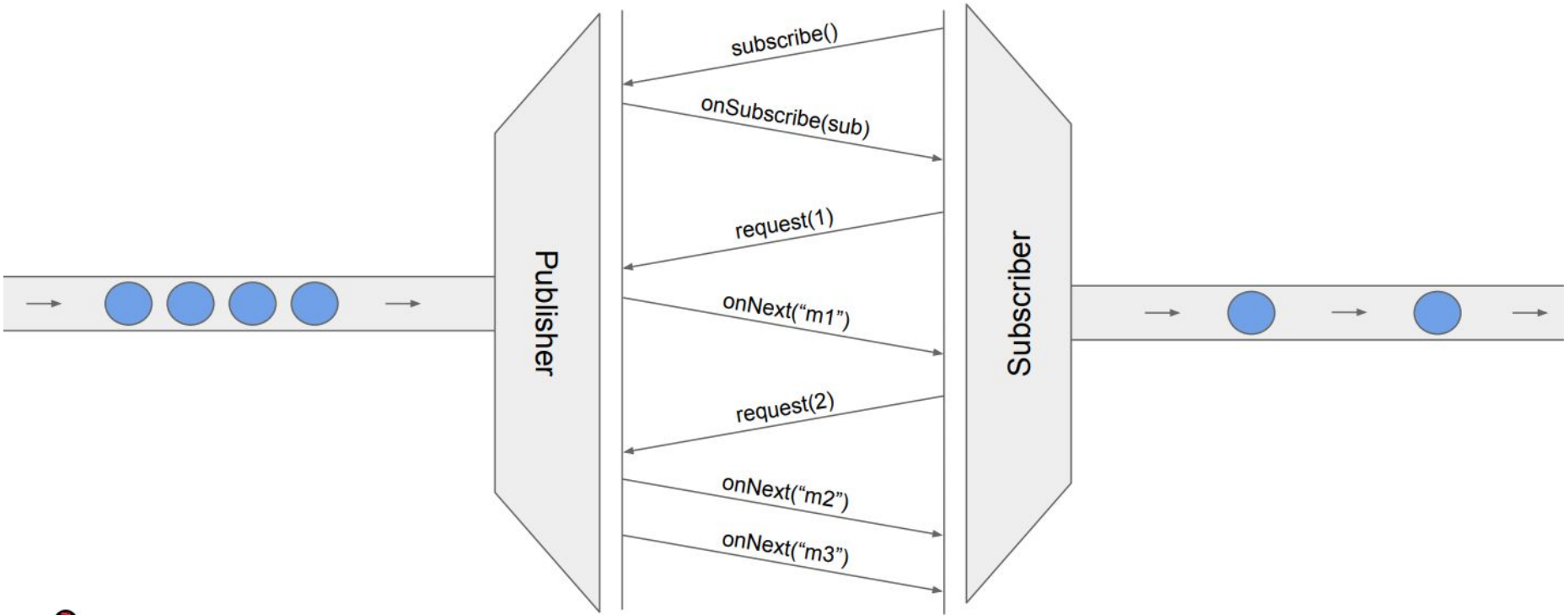
Back Pressure



Back Pressure

With back pressure, the publisher is informed of how many more requests he can send before the buffer overflows.

This avoids an overflow altogether, if the publisher only sends the requested amount of messages.



Akka Streams

Akka Streams is an implementation of Reactive Streams. It has a low level implementation consistent to Reactive Streams, but it also lifts the API to a higher level. On the higher level, the naming is changed.

It is based on Actors. But it is fully typed.

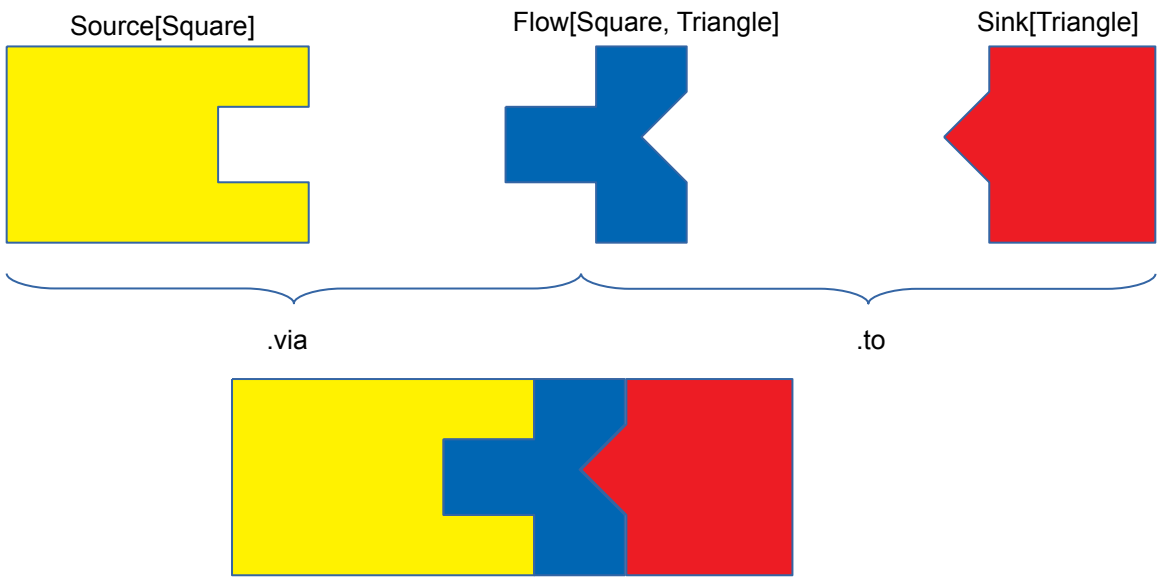
The key abstractions are

- Source
- Flow
- Sink

Akka Streams	Reactive Streams
Source[0, _]	Publisher[0]
Sink[I, _]	Subscriber[I]
Flow[I, 0, _]	Processor[I, 0]

Composition

Source, Flow and Sink can be composed in a type-safe manor.
Connection is through operation via and to



Source[Triangle].via(Flow[Square, Triangle]).to(Sink[Triangle]

Installing Akka Stream

To install, simply add two libraries to your build.sbt file:

```
val akkaVersion = "2.6.17"

libraryDependencies ++= Seq(
  // akka streams
  "com.typesafe.akka" %% "akka-stream" % akkaVersion,
  "com.typesafe.akka" %% "akka-actor-typed" % akkaVersion,
)
```

A simple Example

Here is a simple example that transforms Integers by factor of 2

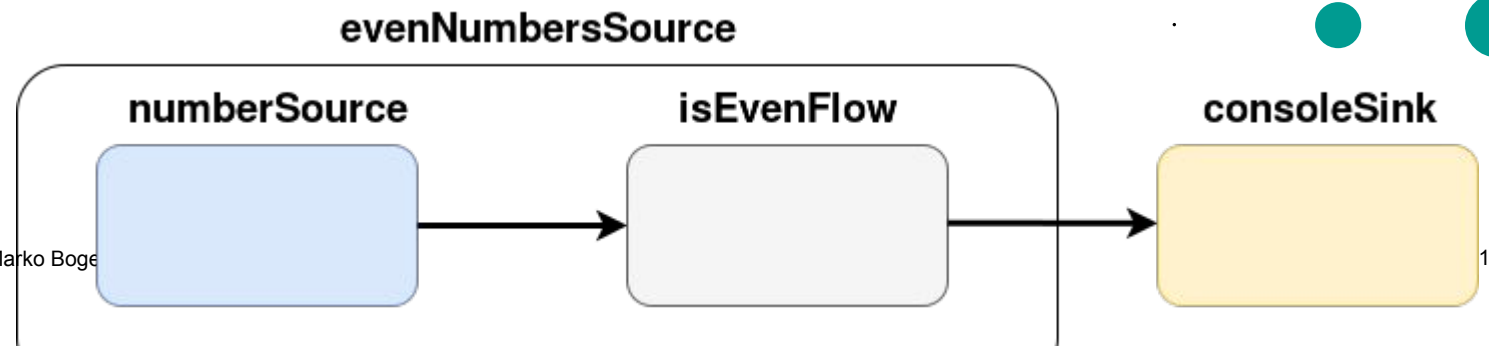
```
implicit val system = ActorSystem()
import system.dispatcher // "thread pool"

// a source element that "emits" integers
val source = Source(1 to 1000)
// a flow that receives integers, transforms them and passes their
results further down
val flow = Flow[Int].map(x => x * 2)
// a sink that receives integers and prints each to the console
val sink = Sink.foreach[Int](println)
// combine all components together in a static graph
val graph = source.via(flow).to(sink)
// start the graph = "materialize" it
graph.run()
```

Another Example

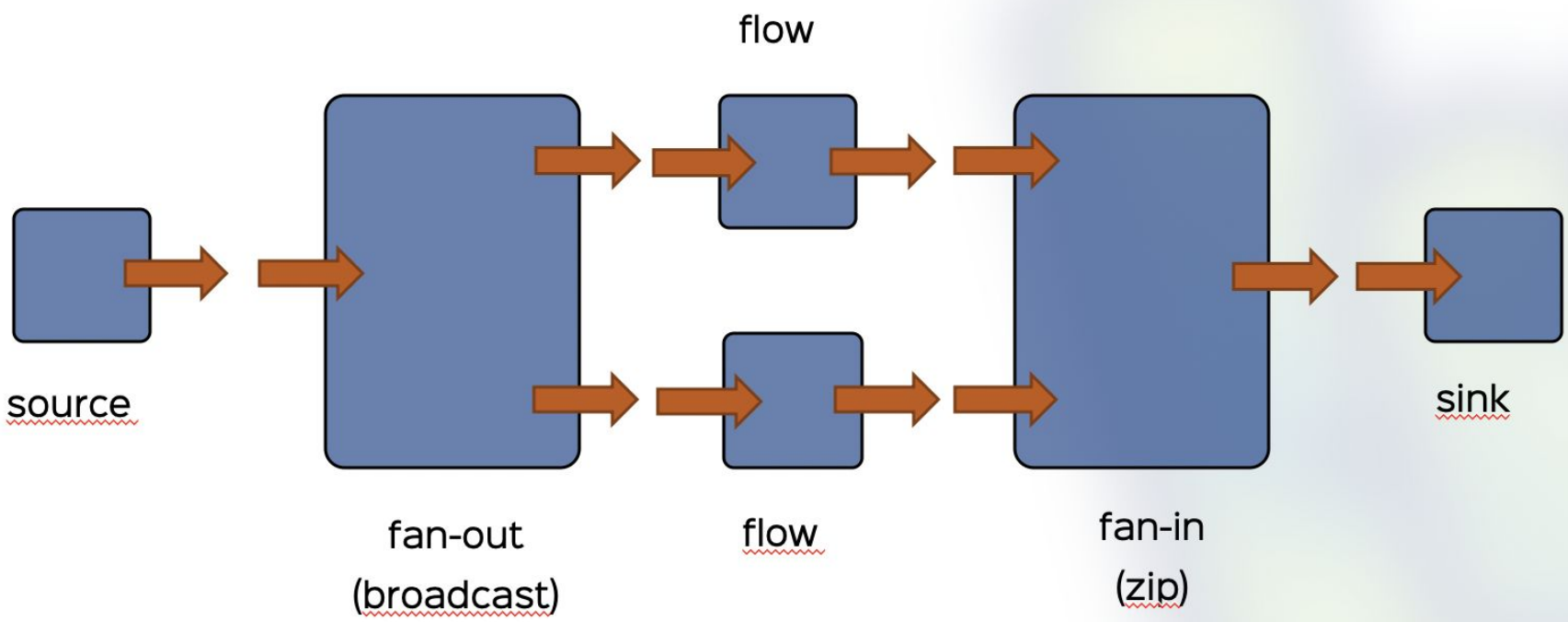
```
object SimpleStreamExample {

 def main(args: Array[String]): Unit = {
  implicit val system       = ActorSystem("Sys")
  implicit val materializer = ActorMaterializer()
  val numbers = 1 to 1000
  //We create a Source that will iterate over the number sequence
  val numberSource: Source[Int, NotUsed] = Source.fromIterator(() => numbers.iterator)
  //Only let pass even numbers through the Flow
  val isEvenFlow: Flow[Int, Int, NotUsed] = Flow[Int].filter((num) => num % 2 == 0)
  //Create a Source of even random numbers by combining the random number Source with the even number filter Flow
  val evenNumbersSource: Source[Int, NotUsed] = numberSource.via(isEvenFlow)
  //A Sink that will write its input onto the console
  val consoleSink: Sink[Int, Future[Done]] = Sink.foreach[Int](println)
  //Connect the Source with the Sink and run it using the materializer
  evenNumbersSource.runWith(consoleSink)
 }
}
```



Graph DSL

The previous mechanims is fine for linear processing. For more complex processes, there is a Graph DSL. It allows a process to fan out and fan in again.



Set up

First, we have to set up an environment for the DSL to work.

```
val graph = GraphDSL.create() { implicit builder: GraphDSL.Builder[NotUsed] =>
  import GraphDSL.Implicits._ // brings some nice operators in scope

}
```

Building Blocks

Then we build the nodes of the graph and add them to the graph.

```
val input = builder.add(Source(1 to 1000)) // the initial elements
val incrementer = builder.add(Flow[Int].map(x => x + 1)) // hard computation #1
val multiplier = builder.add(Flow[Int].map(x => x * 10)) // hard computation #2
val output = builder.add(Sink.foreach[(Int, Int)](println)) // the printer

val broadcast = builder.add(Broadcast[Int](2)) // fan-out operator
val zip = builder.add(Zip[Int, Int]) // fan-in operator
```

Construct the Graph

Now the nodes of the graph are connected, using the ~> operator.

```
input ~> broadcast

broadcast.out(0) ~> incrementer ~> zip.in0
broadcast.out(1) ~> multiplier  ~> zip.in1

zip.out ~> output
```

Closing the Graph Shape and run the Graph

The DSL requires the Graph to be closed.

```
val graph = GraphDSL.create() { implicit builder:
GraphDSL.Builder[NotUsed] =>
  import GraphDSL.Implicits._ // brings some nice operators in scope

    // ... the rest of the implementation

    ClosedShape
}

  def main(args: Array[String]): Unit = {
    RunnableGraph.fromGraph(graph).run()
  }
}
```

Full code can be found here:
https://blog.rockthejvm.com/akka-streams-graphs/

Video here: https://www.youtube.com/watch?v=8XLB28KDtgg

A Complex Graph

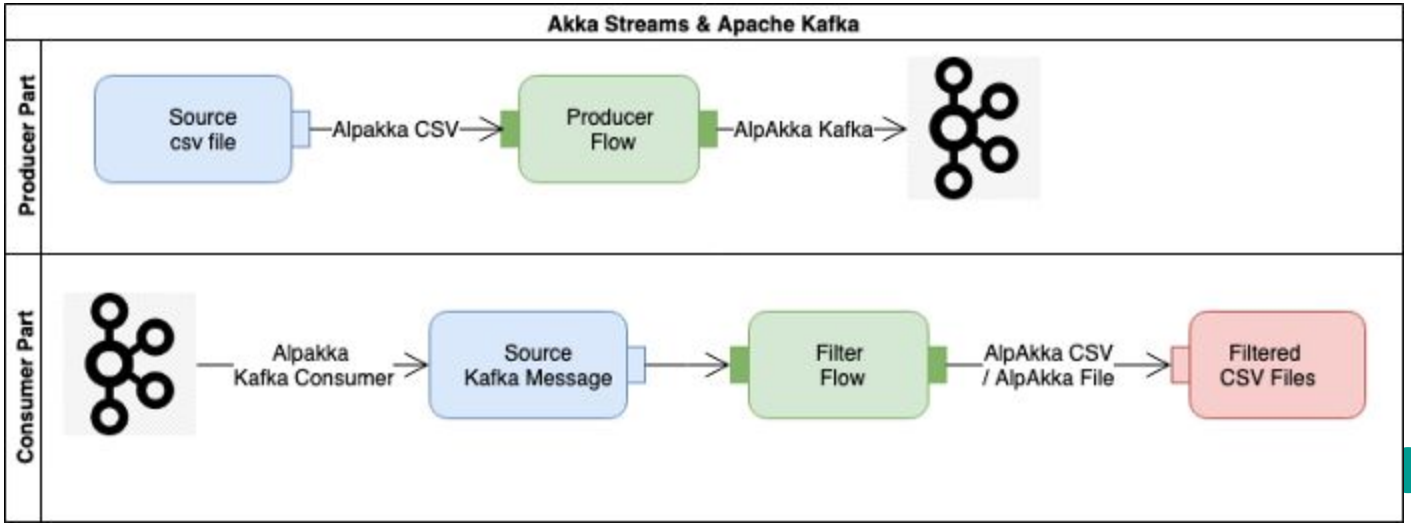
```
import GraphDSL.Implicits._
  RunnableGraph.fromGraph(GraphDSL.create() { implicit builder =>
    val A: Outlet[Int]                  = builder.add(Source.single(0)).out
    val B: UniformFanOutShape[Int, Int] = builder.add(Broadcast[Int](2))
    val C: UniformFanInShape[Int, Int]  = builder.add(Merge[Int](2))
    val D: FlowShape[Int, Int]          = builder.add(Flow[Int].map(_ + 1))
    val E: UniformFanOutShape[Int, Int] = builder.add(Balance[Int](2))
    val F: UniformFanInShape[Int, Int]  = builder.add(Merge[Int](2))
    val G: Inlet[Any]                   = builder.add(Sink.foreach(println)).in


                  C     <~      F
    A  ~>  B  ~>  C     ~>      F
           B  ~>  D  ~>  E  ~>  F
                         E  ~>  G


    ClosedShape
  })
```

Connecting Akka to Other Technologies

The project Alpakka provides connections to a long list of other technologies, for example Kafka



Task: Create an Akka Stream

In your game or application, create a source as reactive stream, create one or more flows to process the input data and finally collect the result in a sink.

Your source could be the keyboard, a file, some web site, random generator or something else. Maybe you can generate data in the form of an external DSL, and parse and process it in a flow.

In the next lectures, we will connect the stream to Kafka.